# AI-Orchestrated Study Companion —Blueprint

> A full-stack, split-screen web application that lets students upload study materials and interact with a personalized AI tutor grounded in their own documents.

---

## Table of Contents


1. Project Overview
2. Tech Stack
3. System Architecture
4. Core Modules
5. Database Schema (3NF)
6. API Design
7. Prompt Orchestration Strategy
8. Frontend Component Tree
9. User Journey
10. Sprint Plan
11. File & Folder Structure
12. Environment Variables
13. Future Roadmap


---

## 1. Project Overview

**Concept:** Students often struggle to extract understanding from dense lecture notes, syllabi, and textbook chapters. This application solves that by pairing the user's own uploaded documents with an AI assistant that answers questions, generates quizzes, and summarises concepts — all grounded exclusively in the provided material.

**Core value propositions:**

- Context-aware AI responses (no hallucinated facts from outside the document)
- Persistent study sessions — pick up exactly where you left off
- Clean, distraction-free split-screen interface
- Streaming AI responses for a real-time, conversational feel

---

## 2. Tech Stack

| Layer | Technology | Rationale |
|---|---|---|
| Frontend | React.js + Tailwind CSS | Component-based UI, utility-first styling |
| Backend | Node.js + Express.js | Fast REST API, excellent streaming support via SSE |
| Database | MySQL (3NF) | Structured relational data, strong FK integrity |
| ORM | Sequelize or `mysql2` | Query builder + migrations |
| Auth | JWT (jsonwebtoken) + bcrypt | Stateless, scalable token auth |
| File uploads | Multer | Multipart form handling in Express |
| AI (cloud) | Google Gemini API | Strong long-context window, generous free tier |
| AI (local) | Ollama (e.g. Mistral, Llama 3) | Zero API cost, privacy-first option |
| Markdown rendering | react-markdown + remark-gfm | Renders AI output with lists, bold, code blocks |
| Streaming | Server-Sent Events (SSE) | Lightweight one-way stream from server to browser |

---

# 3. System Architecture

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              React Frontend                              │
│                Split-Pane Workspace │ Session Dashboard │ Auth UI         │
└──────────────────────────────────────────────────────────────────────────┘
                                │ REST + SSE (JWT in headers)
                                ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                           Node.js + Express Backend                      │
│          Auth Middleware │ Ingestion Engine │ Prompt Orchestrator         │
└──────────────────────────────────────────────────────────────────────────┘
                  │ SQL (mysql2)                          │ HTTPS
                  ▼                                       ▼
┌──────────────────────────┐                  ┌──────────────────────────┐
│  MySQL Database          │                  │        LLM Gateway       │
│  users, sessions,        │                  │   Gemini API / Ollama    │
│  documents, messages     │                  │   (simple RAG inject)    │
└──────────────────────────┘                  └──────────────────────────┘
```

### Request lifecycle (per user message)

1. User sends message → POST /api/chat { message, session_id }
2. Auth middleware → Verify JWT, attach user to req
3. Fetch document text → SELECT content FROM documents WHERE session_id = ?
4. Assemble prompt → "[Doc text] ... Answer: [user question]"
5. Dispatch to LLM → Gemini API or Ollama (streaming enabled)
6. Stream via SSE → Chunks forwarded to React in real time
7. Persist exchange → INSERT INTO messages (role, content, session_id, document_id)

---

# 4. Core Modules

## 4.1 User & Session Management

**Responsibilities:**

- 

```
/*
 * Register and authenticate users (hashed passwords, JWT issuance)
 * Create isolated Study Sessions per topic or exam
 * Track session history and last-accessed timestamp
*/
```

**Key endpoints:**

```
POST /auth/register
```

```
POST /auth/login
GET /sessions
POST /sessions
DELETE /sessions/:id
```

**Implementation notes:**

- 

```
// Passwords hashed with bcrypt (salt rounds: 12)
// JWT expiry: 7 days; refresh token strategy optional for v2
// Sessions are soft-deletable (add deleted_at column) to preserve
message history
```

---

## 4.2 Document Ingestion Engine

**Responsibilities:**

- 

```
/*
 * Accept file uploads from the user via multipart form
 * Extract raw text from the file
 * Store metadata and content in the database linked to the active
session
*/
```

**Supported formats (v1):**

- 

```
.txt — direct read
.md — direct read (strip Markdown syntax optionally)
```

**Future formats:**

- 

```
.pdf — via pdf-parse library
.docx — via mammoth
```

**Key endpoints:**

```
POST /sessions/:id/documents
GET /sessions/:id/documents
DELETE /documents/:id
```

**Implementation notes:**

- 

```
// Use Multer with memStorage for v1 (files read into memory, text
extracted, file discarded — no disk storage needed initially)
// Store content as LONGTEXT in MySQL to handle large documents
// Cap upload size at 5 MB initially; raise when chunked RAG is
implemented
```

---

## 4.3 Prompt Orchestration Engine

This is the brain of the application. The backend mediates between the React chat and the external LLM.

**Simple RAG pattern (v1 — full document injection):**

System prompt:

```
"You are a focused study assistant. Answer questions ONLY using the
   provided study material. If the answer is not in the material, say
so."
```

User turn:

```
"Here is the study material:
   ---
   {document.content}
   ---
   Student question: {user.message}"
```

**Streaming implementation (SSE):**

```
// Express SSE route skeleton
res.setHeader('Content-Type', 'text/event-stream');
res.setHeader('Cache-Control', 'no-cache');
res.setHeader('Connection', 'keep-alive');

for await (const chunk of llmStream) {
  res.write(`data: ${JSON.stringify({ text: chunk })}\n\n`);
}

res.write('data: [DONE]\n\n');
res.end();
```

**LLM adapter pattern** — abstract behind an interface so Gemini and Ollama are interchangeable:

```
// llm/index.js
const provider = process.env.LLM_PROVIDER === 'ollama' ? ollamaAdapter
: geminiAdapter;
export const streamCompletion = (prompt) => provider.stream(prompt);
```

---

## 4.4 Interactive Workspace (Frontend)

**Split-pane layout:**

```
┌──────────────────────────┬──────────────────────────┐
│   Document Viewer        │   AI Chat Interface      │
│   (left, read-only)      │   (right, interactive)   │
│                          │                          │
│   Raw text or            │   Message bubbles        │
│   Markdown render        │   react-markdown output  │
│   of uploaded doc        │   Streaming indicator    │
│                          │                          │
│                          │   Quick actions:         │
│                          │   [Summarise] [Flashcards] │
│                          │   [Explain simply] [Quiz]  │
└──────────────────────────┴──────────────────────────┘
```

**Quick action prompts:**

| Button | Injected prompt |
|---|---|
| Summarise | "Summarise the key points from this document in bullet form." |
| Flashcards | "Generate 5 flashcard pairs (Q&A) from this document." |
| Explain simply | "Explain the main concept in this document as if I'm a beginner." |
| Quiz me | "Create a 5-question multiple-choice quiz based on this document." |

---

# 5. Database Schema (3NF)

## Entity Relationship Overview

users ──< study_sessions ──< documents
└──< messages >── documents (FK nullable)

## Table definitions

**users**

```
CREATE TABLE users (
  id            CHAR(36)      PRIMARY KEY DEFAULT (UUID()),
  email         VARCHAR(255)  NOT NULL UNIQUE,
  password_hash VARCHAR(255)  NOT NULL,
  created_at    TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

**study_sessions**

```
CREATE TABLE study_sessions (
  id            CHAR(36)      PRIMARY KEY DEFAULT (UUID()),
  user_id       CHAR(36)      NOT NULL,
  title         VARCHAR(255)  NOT NULL,
  created_at    TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  last_accessed TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP ON
UPDATE CURRENT_TIMESTAMP,
  deleted_at    TIMESTAMP     NULL,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

**documents**

```sql
CREATE TABLE documents (
  id           CHAR(36)      PRIMARY KEY DEFAULT (UUID()),
  session_id   CHAR(36)      NOT NULL,
  filename     VARCHAR(255)  NOT NULL,
  mime_type    VARCHAR(100)  NOT NULL,
  content      LONGTEXT      NOT NULL,
  uploaded_at  TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (session_id) REFERENCES study_sessions(id) ON DELETE
CASCADE
);
```

**messages**

```sql
CREATE TABLE messages (
  id           CHAR(36)      PRIMARY KEY DEFAULT (UUID()),
  session_id   CHAR(36)      NOT NULL,
  document_id  CHAR(36)      NULL,
  role         ENUM('user','assistant') NOT NULL,
  content      TEXT          NOT NULL,
  created_at   TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (session_id)  REFERENCES study_sessions(id) ON DELETE
CASCADE,
  FOREIGN KEY (document_id) REFERENCES documents(id) ON DELETE SET
NULL
);
```

---

# 6. API Design

### Auth

| Method | Path | Body | Response |
|---|---|---|---|
| POST | /auth/register | { email, password } | { token, user } |
| POST | /auth/login | { email, password } | { token, user } |

### Sessions

| Method | Path | Auth | Response |
|---|---|---|---|
| GET | /api/sessions | ✅ | Array of session objects |
| POST | /api/sessions | ✅ | Created session |
| DELETE | /api/sessions/:id | ✅ | 204 No Content |

### Documents

| Method | Path | Auth | Body | Response |
|---|---|---|---|---|
| POST | /api/sessions/:id/documents | ✅ | multipart/form-data file | Created document metadata |
| GET | /api/sessions/:id/documents | ✅ | — | Array of document metadata |
| DELETE | /api/documents/:id | ✅ | — | 204 No Content |

### Chat

| Method | Path | Auth | Body | Response |
|---|---|---|---|---|
| POST | /api/chat | ✅ | { session_id, document_id, message } | SSE stream |
| GET | /api/sessions/:id/messages | ✅ | — | Array of messages |

---

# 7. Prompt Orchestration Strategy

This section defines the two main strategies for ensuring the AI assistant uses the uploaded study material to answer questions.

## v1 — Full document injection (MVP)

This strategy is suitable for documents up to ~30,000 tokens (approx. 20–25 pages of text).
**Process Flow:**

1. **System Prompt:** The AI is initialized with the instruction: `[System] You are a focused study assistant. Answer questions ONLY using the provided study material. If the answer is not in the material, say so.`
2. **User Prompt Assembly:** The user's question is combined with the entire document content to form the final prompt sent to the LLM:

```
[User] Study material:
---
{full document text}
---
Question: {user message}
```

- **Pros:** Simple, no vector DB needed, highly accurate for single-document sessions.
- **Cons:** Token limit hit on large documents; cost scales with document size per request.

## v2 — Chunked retrieval (future)

This strategy is used when documents exceed the context window.
**On Upload:**

- Split document into ~500-token overlapping chunks
- Embed each chunk via an embedding model (e.g. text-embedding-004)
- Store vectors in a vector store (pgvector, Chroma, or Pinecone)

**At Query Time:**

- At query time, retrieve top-k most relevant chunks via cosine similarity
- Inject only the retrieved chunks into the prompt

---

# 8. Frontend Component Tree

```
<App>
├── <AuthProvider>        (JWT context, login/logout)
├── <Router>
│   ├── /login            → <LoginPage>
│   ├── /register         → <RegisterPage>
│   ├── /dashboard        → <DashboardPage>
│   │   └── <SessionCard>[] (list of study sessions)
│   └── /sessions/:id     → <WorkspacePage>
│       ├── <DocumentPane>  (left — read-only document viewer)
│       │   └── <MarkdownRenderer>
│       └── <ChatPane>      (right — interactive chat)
│           ├── <MessageList>
│           │   └── <MessageBubble>[] (role: user | assistant)
│           │       └── <MarkdownRenderer> (for assistant messages)
│           ├── <StreamingIndicator>  (animated dots while AI responds)
│           ├── <QuickActions>        (Summarise, Flashcards, etc.)
│           └── <ChatInput>           (textarea + send button)
```

### Key React patterns

- **Context:** `AuthContext` for token + user, `SessionContext` for active session state
- **Custom hooks:** `useChat(sessionId)` — manages message list, SSE connection, send handler
- **SSE hook:** Opens `EventSource` on chat submission, appends chunks to assistant bubble in real time, closes on `[DONE]`

---

## 9. User Journey

```
1. Register / Login
   └── Land on Dashboard

2. Click "New Study Session"
   └── Enter title → "History Final Exam"
   └── Session created, navigate to Workspace

3. Upload document
   └── Click "Upload notes" → select notes.txt
   └── Backend extracts text, stores in DB
   └── Document appears in left pane

4. Ask a question
   └── Type: "What were the main causes of the war?"
   └── POST /api/chat → SSE stream opens
   └── AI answer streams into right pane in real time

5. Use quick actions
   └── Click [Flashcards] → 5 Q&A pairs generated
   └── Click [Quiz me] → 5 MCQ questions appear

6. Session saved automatically
   └── Close browser, return tomorrow
   └── Dashboard shows session with last-accessed time
   └── All chat history and document preserved
```

---

# 10. Sprint Plan

## Sprint 1 — Foundation

**Goal:** Running skeleton with auth and database

- [ ] Initialise Express app with folder structure
- [ ] Connect MySQL, create all four tables via migration scripts
- [ ] Implement `POST /auth/register` and `POST /auth/login`
- [ ] Implement JWT middleware (protect all `/api` routes)
- [ ] Basic React app scaffold with React Router
- [ ] Login and Register pages with form validation
- [ ] Auth context + protected route wrapper
- [ ] End-to-end test: register → login → receive JWT

---

## Sprint 2 — Sessions & Document Ingestion

**Goal:** Users can create sessions and upload documents

- [ ] `GET/POST/DELETE /api/sessions` endpoints
- [ ] Dashboard page — list and create sessions
- [ ] `POST /api/sessions/:id/documents` with Multer
- [ ] Text extraction from `.txt` and `.md` files
- [ ] Document stored in DB, metadata returned to client
- [ ] Workspace page scaffolded (split-pane layout)
- [ ] Document rendered in left pane on load
- [ ] End-to-end test: create session → upload file → see text in pane

---

## Sprint 3 — AI Chat & Streaming

**Goal:** Working AI chat grounded in the uploaded document

- [ ] LLM adapter module (Gemini + Ollama implementations)
- [ ] Prompt assembly — inject document text + user question
- [ ] `POST /api/chat` with SSE streaming response
- [ ] `GET /api/sessions/:id/messages` for history reload
- [ ] Chat pane UI — message bubbles, `react-markdown` rendering
- [ ] SSE `useChat` hook — opens EventSource, streams chunks into bubble
- [ ] Streaming indicator (animated dots)
- [ ] Persist both user message and assistant response to DB
- [ ] End-to-end test: ask question → streamed answer → verify DB record

---

## Sprint 4 — Polish & Quick Actions (Week 7–8)

**Goal:** Production-ready UX

- [ ] Quick action buttons (Summarise, Flashcards, Explain simply, Quiz me)
- [ ] Error handling — expired JWT redirect, upload failures, LLM errors
- [ ] Loading states for all async operations
- [ ] Responsive layout adjustments (tablet support)
- [ ] Session last-accessed timestamp update on open
- [ ] Delete session (soft delete) from dashboard
- [ ] Input validation on all forms (email format, password length, file type)

- [ ] Environment config — `.env.example` documented
- [ ] README with setup instructions

---

## 11. File & Folder Structure

```
study-companion/
├── client/                         # React frontend
│   ├── public/
│   ├── src/
│   │   ├── components/
│   │   │   ├── auth/
│   │   │   │   ├── LoginForm.jsx
│   │   │   │   └── RegisterForm.jsx
│   │   │   ├── dashboard/
│   │   │   │   └── SessionCard.jsx
│   │   │   └── workspace/
│   │   │       ├── DocumentPane.jsx
│   │   │       ├── ChatPane.jsx
│   │   │       ├── MessageBubble.jsx
│   │   │       ├── QuickActions.jsx
│   │   │       ├── ChatInput.jsx
│   │   │       └── StreamingIndicator.jsx
│   │   ├── context/
│   │   │   └── AuthContext.jsx
│   │   ├── hooks/
│   │   │   └── useChat.js
│   │   ├── pages/
│   │   │   ├── LoginPage.jsx
│   │   │   ├── RegisterPage.jsx
│   │   │   ├── DashboardPage.jsx
│   │   │   └── WorkspacePage.jsx
│   │   ├── services/
│   │   │   └── api.js              # Axios instance + request helpers
│   │   └── App.jsx
│   └── package.json
│
├── server/                         # Express backend
```

```
│   ├── config/
│   │   └── db.js                   # MySQL connection pool
│   ├── controllers/
│   │   ├── authController.js
│   │   ├── sessionController.js
│   │   ├── documentController.js
│   │   └── chatController.js
│   ├── llm/
│   │   ├── index.js                # Adapter selector
│   │   ├── geminiAdapter.js
│   │   └── ollamaAdapter.js
│   ├── middleware/
│   │   ├── authMiddleware.js       # JWT verify
│   │   └── uploadMiddleware.js     # Multer config
│   ├── migrations/
│   │   └── 001_init.sql
│   ├── routes/
│   │   ├── auth.js
│   │   ├── sessions.js
│   │   ├── documents.js
│   │   └── chat.js
│   ├── utils/
│   │   └── extractText.js          # txt/md/pdf text extraction
│   ├── app.js
│   └── package.json
│
├── .env.example
└── README.md
```

---

## 12. Environment Variables

**Server Configuration (.env)**

```
# Network configuration for the Express server
PORT=4000
CLIENT_URL=http://localhost:5173

# MySQL database connection details
```

```
DB_HOST=localhost
DB_PORT=3306
DB_NAME=study_companion
DB_USER=root
DB_PASSWORD=your_secure_password

# Authentication settings (JWT)
JWT_SECRET=use_a_long_random_string_here
JWT_EXPIRES_IN=7d

# Artificial Intelligence Provider (gemini or ollama)
LLM_PROVIDER=gemini
GEMINI_API_KEY=your_google_gemini_api_key
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=mistral
```

**Client Configuration (.env)**

```
# Base URL for the backend API consumed by Vite
VITE_API_BASE_URL=http://localhost:4000
```

---

# 13. Future Roadmap

### v1.1 — PDF & DOCX support

- Add `pdf-parse` for `.pdf` extraction
- Add `mammoth` for `.docx` extraction
- Auto-detect MIME type and route to the correct extractor

### v1.2 — Multi-document sessions

- Allow multiple documents per session
- Chat interface lets user select which document(s) to query
- Inject multiple documents or let user switch context

### v2.0 — Vector RAG

- Chunk documents on upload (500-token overlapping chunks)
- Embed chunks via Gemini `text-embedding-004` or `nomic-embed-text` (Ollama)
- Store vectors in pgvector or Chroma
- At query time: embed question, retrieve top-5 chunks, inject only those

## v2.1 — Highlight & cite

- AI response includes source chunk references
- Document pane highlights the passages used to construct each answer

## v3.0 — Collaborative sessions

- Share a session with a classmate via invite link
- Real-time collaborative chat using WebSockets
- Shared flashcard decks and quiz scores

---

*AI-Orchestrated Study Companion project.*
*Stack: React · Node/Express · MySQL · Gemini API*